



中国海洋大学  
OCEAN UNIVERSITY OF CHINA

# 第三周实验报告

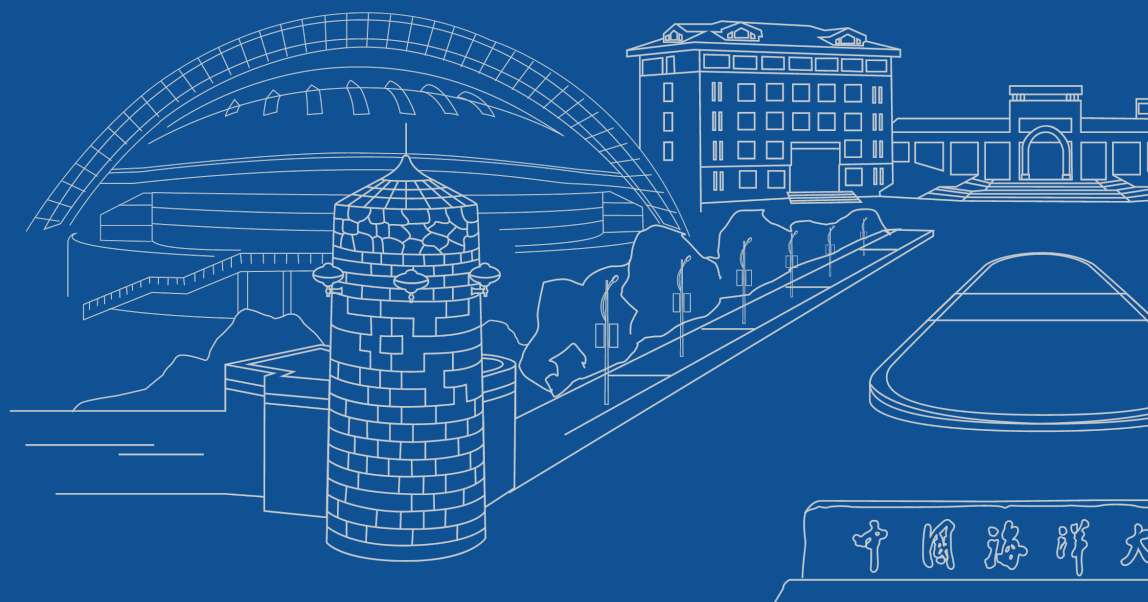
系统开发工具基础

刘晨希

25020007076

信息科学与工程学部  
计算机科学与技术学院  
计算机科学与技术专业  
系统开发工具基础

2026 年 9 月 9 日



中国海洋大学



中国海洋大学  
OCEAN UNIVERSITY OF CHINA

# 第三周实验报告

系统开发工具基础

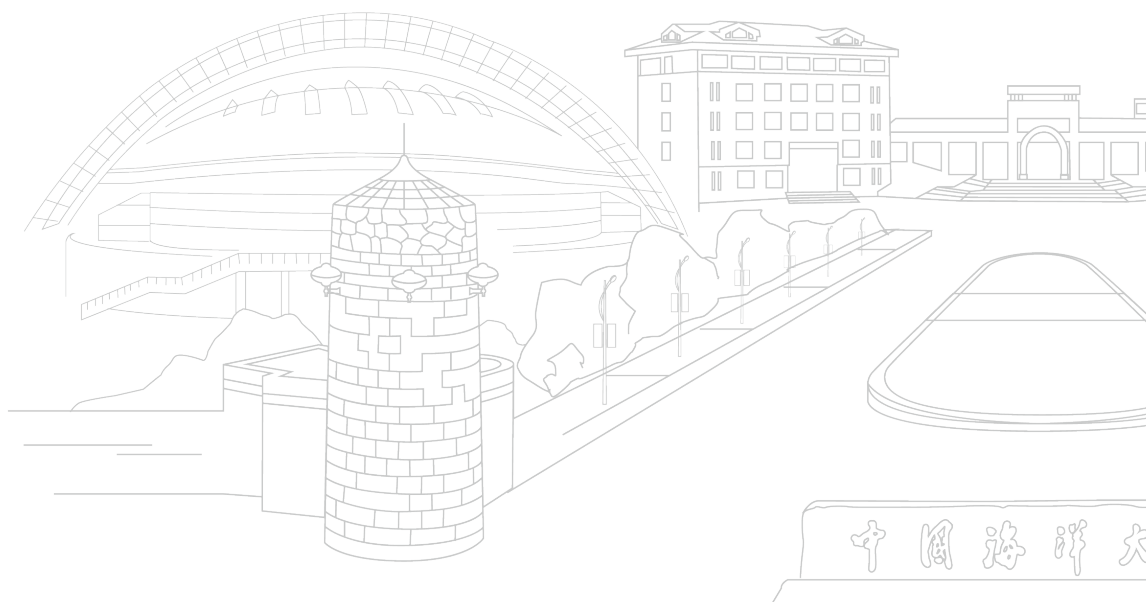
刘晨希

25020007076

指导老师: 周小伟

信息科学与工程学部  
计算机科学与技术学院  
计算机科学与技术专业  
系统开发工具基础

2026 年 9 月 9 日



# 目录

|                                                      |           |
|------------------------------------------------------|-----------|
| 目录                                                   | i         |
| <b>1 课堂练习及结果</b>                                     | <b>2</b>  |
| 1.1 第 9 题从源码构建并在干净环境安装 <b>Wheel</b>                  | 2         |
| 1.1.1 实验内容                                           | 2         |
| 1.1.2 实验结果                                           | 5         |
| 1.2 第 10 题让编程智能体进入可验证的修复循环                           | 5         |
| 1.2.1 实验内容                                           | 5         |
| 1.2.2 实验结果                                           | 8         |
| 1.3 第 11 题把“无法处理”的协作材料改成可执行信息                        | 8         |
| 1.3.1 实验内容                                           | 8         |
| 1.3.2 实验结果                                           | 9         |
| 1.4 第 12 题修复一个可复现的线性回归训练循环                           | 9         |
| 1.4.1 实验内容                                           | 9         |
| 1.4.2 实验结果                                           | 11        |
| 1.5 Github 仓库地址                                      | 11        |
| <b>2 课后练习内容和结果</b>                                   | <b>12</b> |
| 2.1 基础命令与重定向                                         | 12        |
| 题 1: 确认 <b>Shell</b> 环境                              | 12        |
| 题 2: <b>ls -l</b> 的输出格式                              | 12        |
| 题 3: <b>glob</b> 模式匹配                                | 13        |
| 题 4: 三种引号的区别                                         | 14        |
| 题 5: 标准流与重定向                                         | 14        |
| 题 6: 退出状态与 <b>&amp;&amp;/   </b>                     | 15        |
| 2.2 脚本编写与权限                                          | 16        |
| 题 7: 为什么 <b>cd</b> 必须是 <b>Shell</b> 内建命令?            | 16        |
| 题 8: 写脚本检查文件是否存在                                     | 16        |
| 题 9: <b>chmod +x</b> 的作用                             | 17        |
| 题 10: <b>set -x</b> 的作用                              | 18        |
| 题 11: 带日期的备份文件                                       | 19        |
| 题 12: 修改 <b>flaky test</b> 脚本接收命令行参数                 | 19        |
| 2.3 管道、工具与进阶                                         | 21        |
| 题 13: 找出 <b>home</b> 目录最常见的 5 种文件扩展名                 | 21        |
| 题 14: <b>xargs</b> 结合 <b>find</b> 统计 <b>.sh</b> 文件行数 | 21        |

---

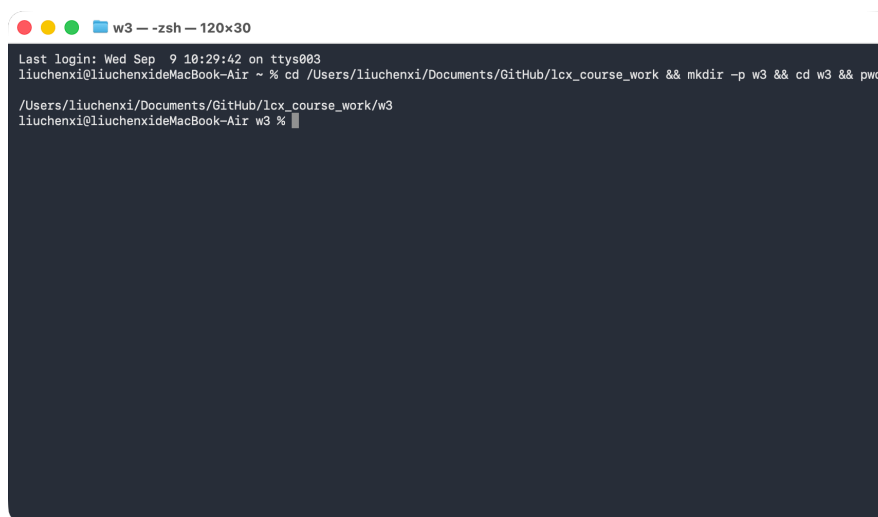
|                                            |           |
|--------------------------------------------|-----------|
| 题 15: curl 获取课程网站 HTML, 统计列出了多少讲 . . . . . | 22        |
| 题 16: jq 处理 JSON . . . . .                 | 23        |
| 题 17: awk 按列过滤并交换列 . . . . .               | 24        |
| 题 18: 拆解 SSH 日志管道 + 从历史记录找出最常用命令 . . . . . | 25        |
| <b>3 解题感悟</b>                              | <b>27</b> |

## 课堂练习及结果

### 1.1 第 9 题从源码构建并在干净环境安装 Wheel

#### 1.1.1 实验内容

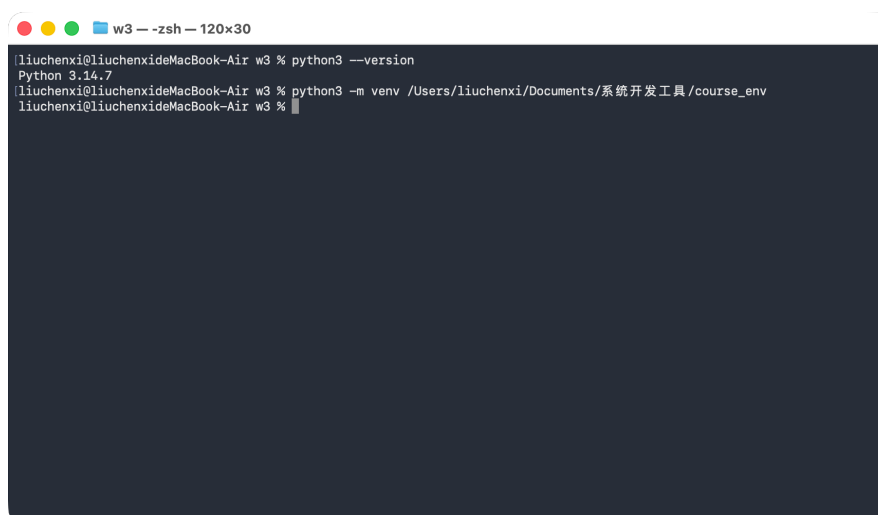
如图 1.1 所示，在课程仓库中创建 w3 实验目录并进入，第三周的四个实验均在 w3 目录下完成。

A terminal window titled 'w3 --zsh -- 120x30' showing the following commands and output:

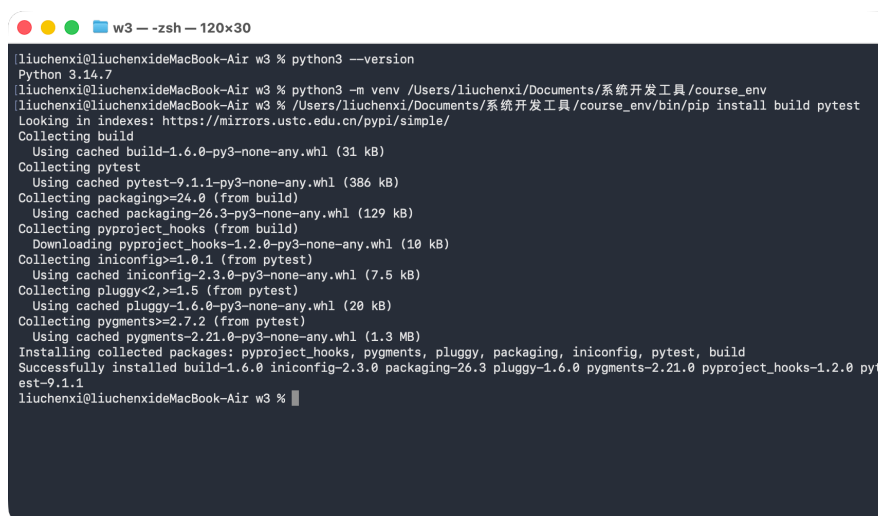
```
Last login: Wed Sep  9 18:29:42 on ttys003
liuchenxi@liuchenxiMacBook-Air ~ % cd /Users/liuchenxi/Documents/GitHub/lcx_course_work && mkdir -p w3 && cd w3 && pwd
/Users/liuchenxi/Documents/GitHub/lcx_course_work/w3
liuchenxi@liuchenxiMacBook-Air w3 %
```

图 1.1: 创建 w3 实验目录

由于本机没有预装课程要求的构建工具，需要先准备课程环境。如图 1.2 所示，先检查 Python 版本 (3.14.7)，再使用 `python3 -m venv` 命令创建课程虚拟环境 `course_env`；如图 1.3 所示，在 `course_env` 中执行 `pip install build pytest` 安装构建和测试工具，为后续构建 wheel 和运行测试做准备。



```
w3 -- zsh -- 120x30
liuchenxi@liuchenxideMacBook-Air w3 % python3 --version
Python 3.14.7
liuchenxi@liuchenxideMacBook-Air w3 % python3 -m venv /Users/liuchenxi/Documents/系统开发工具/course_env
liuchenxi@liuchenxideMacBook-Air w3 %
```

图 1.2: 检查 Python 版本并创建课程虚拟环境 `course_env`

```
w3 -- zsh -- 120x30
liuchenxi@liuchenxideMacBook-Air w3 % python3 --version
Python 3.14.7
liuchenxi@liuchenxideMacBook-Air w3 % python3 -m venv /Users/liuchenxi/Documents/系统开发工具/course_env
liuchenxi@liuchenxideMacBook-Air w3 % /Users/liuchenxi/Documents/系统开发工具/course_env/bin/pip install build pytest
Looking in indexes: https://mirrors.ustc.edu.cn/pypi/simple/
Collecting build
  Using cached build-1.6.0-py3-none-any.whl (31 kB)
Collecting pytest
  Using cached pytest-9.1.1-py3-none-any.whl (386 kB)
Collecting packaging>=24.0 (from build)
  Using cached packaging-26.3-py3-none-any.whl (129 kB)
Collecting pyproject_hooks (from build)
  Downloading pyproject_hooks-1.2.0-py3-none-any.whl (10 kB)
Collecting iniconfig>=1.0.1 (from pytest)
  Using cached iniconfig-2.3.0-py3-none-any.whl (7.5 kB)
Collecting pluggy<2,>=1.5 (from pytest)
  Using cached pluggy-1.6.0-py3-none-any.whl (20 kB)
Collecting pygments>=2.7.2 (from pytest)
  Using cached pygments-2.21.0-py3-none-any.whl (1.3 MB)
Installing collected packages: pyproject_hooks, pygments, pluggy, packaging, iniconfig, pytest, build
Successfully installed build-1.6.0 iniconfig-2.3.0 packaging-26.3 pluggy-1.6.0 pygments-2.21.0 pyproject_hooks-1.2.0 pyt
est-9.1.1
liuchenxi@liuchenxideMacBook-Air w3 %
```

图 1.3: 在 `course_env` 中安装 `build` 与 `pytest`

如图 1.4 所示，在 q09 中创建 `src/greetlab/__init__.py`、`cli.py` 和 `pyproject.toml`。其中 `[project]` 段的 `name` 为 `greetlab-25020007076`（把题目模板中的“学号”替换为自己的学号）、`version` 为 `0.1.0`；`[project.scripts]` 段把命令行入口 `sdt-greet` 指向 `greetlab.cli` 的 `main` 函数。

```
w3 -- zsh -- 120x30

requires = ["setuptools>=68"]
build-backend = "setuptools.build_meta"

[project]
name = "greetlab-25020007076"
version = "0.1.0"

[project.scripts]
sdt-greet = "greetlab.cli:main"
EOF
liuchenxi@liuchenxiMacBook-Air w3 % cat q09/src/greetlab/cli.py
import argparse

def main():
    p = argparse.ArgumentParser()
    p.add_argument("--name", required=True)
    a = p.parse_args()
    print(f"Hello, {a.name}!")
liuchenxi@liuchenxiMacBook-Air w3 % cat q09/pyproject.toml
[build-system]
requires = ["setuptools>=68"]
build-backend = "setuptools.build_meta"

[project]
name = "greetlab-25020007076"
version = "0.1.0"

[project.scripts]
sdt-greet = "greetlab.cli:main"
liuchenxi@liuchenxiMacBook-Air w3 %
```

图 1.4: 创建 cli.py 与 pyproject.toml

如图 1.5 所示, 激活 course\_env 后运行 `python -m build`, 成功生成 `greetlab_25020007076-0.1.0.tar.gz` 和 `greetlab_25020007076-0.1.0-py3-none-any.whl` 两个分发产物。

```
q09 -- zsh -- 120x30

writing src/greetlab_25020007076.egg-info/PKG-INFO
writing dependency_links to src/greetlab_25020007076.egg-info/dependency_links.txt
writing entry points to src/greetlab_25020007076.egg-info/entry_points.txt
writing top-level names to src/greetlab_25020007076.egg-info/top_level.txt
reading manifest file 'src/greetlab_25020007076.egg-info/SOURCES.txt'
writing manifest file 'src/greetlab_25020007076.egg-info/SOURCES.txt'
installing to build/bdist.macosx-26.0-arm64/wheel
running install
running install_lib
creating build/bdist.macosx-26.0-arm64/wheel
creating build/bdist.macosx-26.0-arm64/wheel/greetlab
copying build/lib/greetlab/__init__.py -> build/bdist.macosx-26.0-arm64/wheel/./greetlab
copying build/lib/greetlab/cli.py -> build/bdist.macosx-26.0-arm64/wheel/./greetlab
running install_egg_info
Copying src/greetlab_25020007076.egg-info to build/bdist.macosx-26.0-arm64/wheel/./greetlab_25020007076-0.1.0-py3.14.egg-info
running install_scripts
creating build/bdist.macosx-26.0-arm64/wheel/greetlab_25020007076-0.1.0.dist-info/WHEEL
creating '/Users/liuchenxi/Documents/GitHub/lcx_course_work/w3/q09/dist/.tmp-la7trg74/greetlab_25020007076-0.1.0-py3-none-any.whl' and adding 'build/bdist.macosx-26.0-arm64/wheel' to it
adding 'greetlab/__init__.py'
adding 'greetlab/cli.py'
adding 'greetlab_25020007076-0.1.0.dist-info/METADATA'
adding 'greetlab_25020007076-0.1.0.dist-info/WHEEL'
adding 'greetlab_25020007076-0.1.0.dist-info/entry_points.txt'
adding 'greetlab_25020007076-0.1.0.dist-info/top_level.txt'
adding 'greetlab_25020007076-0.1.0.dist-info/RECORD'
removing build/bdist.macosx-26.0-arm64/wheel
Successfully built greetlab_25020007076-0.1.0.tar.gz and greetlab_25020007076-0.1.0-py3-none-any.whl
(course_env) liuchenxi@liuchenxiMacBook-Air q09 %
```

图 1.5: 运行 `python -m build` 生成 wheel

如图 1.6 所示, 先用 `ls dist` 查看构建产物, 再使用 `python -m venv venv` 新建一个干净虚拟环境, 最后用 `venv/bin/pip install dist/greetlab_25020007076-0.1.0-py3-none-any.whl` 只从生成的 wheel 安装, 输出 `Successfully installed greetlab-25020007076-0.1.0`, 全程没有从镜像源下载安装该包。

```

q09 --zsh -- 120x30
creating build/bdist.macosx-26.0-arm64/wheel
creating build/bdist.macosx-26.0-arm64/wheel/greetlab
copying build/lib/greetlab/__init__.py -> build/bdist.macosx-26.0-arm64/wheel/./greetlab
copying build/lib/greetlab/cli.py -> build/bdist.macosx-26.0-arm64/wheel/./greetlab
running install_egg_info
Copying src/greetlab_25020007076.egg-info to build/bdist.macosx-26.0-arm64/wheel/./greetlab_25020007076-0.1.0-py3.14.egg-info
running install_scripts
creating build/bdist.macosx-26.0-arm64/wheel/greetlab_25020007076-0.1.0.dist-info/WHEEL
creating '/Users/liuchenxi/Documents/GitHub/lcx_course_work/w3/q09/dist/.tmp-la7trg74/greetlab_25020007076-0.1.0-py3-none-any.whl' and adding 'build/bdist.macosx-26.0-arm64/wheel' to it
adding 'greetlab/__init__.py'
adding 'greetlab/cli.py'
adding 'greetlab_25020007076-0.1.0.dist-info/METADATA'
adding 'greetlab_25020007076-0.1.0.dist-info/WHEEL'
adding 'greetlab_25020007076-0.1.0.dist-info/entry_points.txt'
adding 'greetlab_25020007076-0.1.0.dist-info/top_level.txt'
adding 'greetlab_25020007076-0.1.0.dist-info/RECORD'
removing build/bdist.macosx-26.0-arm64/wheel
Successfully built greetlab_25020007076-0.1.0.tar.gz and greetlab_25020007076-0.1.0-py3-none-any.whl
(course_env) liuchenxi@liuchenxideMacBook-Air q09 % ls dist
greetlab_25020007076-0.1.0-py3-none-any.whl    greetlab_25020007076-0.1.0.tar.gz
(course_env) liuchenxi@liuchenxideMacBook-Air q09 % python -m venv venv
(course_env) liuchenxi@liuchenxideMacBook-Air q09 % venv/bin/pip install dist/greetlab_25020007076-0.1.0-py3-none-any.whl
1
Looking in indexes: https://mirrors.ustc.edu.cn/pypi/simple/
Processing ./dist/greetlab_25020007076-0.1.0-py3-none-any.whl
Installing collected packages: greetlab-25020007076
Successfully installed greetlab-25020007076-0.1.0
(course_env) liuchenxi@liuchenxideMacBook-Air q09 %

```

图 1.6: 新建干净虚拟环境并仅从 wheel 安装

如图 1.7 所示, 切换到 q09 之外 (w3 目录) 运行 `sdt-greet --name 25020007076`, 输出 Hello, 25020007076!, 说明 wheel 安装正确、命令行入口可用。

```

w3 --zsh -- 120x30
copying build/lib/greetlab/cli.py -> build/bdist.macosx-26.0-arm64/wheel/./greetlab
running install_egg_info
Copying src/greetlab_25020007076.egg-info to build/bdist.macosx-26.0-arm64/wheel/./greetlab_25020007076-0.1.0-py3.14.egg-info
running install_scripts
creating build/bdist.macosx-26.0-arm64/wheel/greetlab_25020007076-0.1.0.dist-info/WHEEL
creating '/Users/liuchenxi/Documents/GitHub/lcx_course_work/w3/q09/dist/.tmp-la7trg74/greetlab_25020007076-0.1.0-py3-none-any.whl' and adding 'build/bdist.macosx-26.0-arm64/wheel' to it
adding 'greetlab/__init__.py'
adding 'greetlab/cli.py'
adding 'greetlab_25020007076-0.1.0.dist-info/METADATA'
adding 'greetlab_25020007076-0.1.0.dist-info/WHEEL'
adding 'greetlab_25020007076-0.1.0.dist-info/entry_points.txt'
adding 'greetlab_25020007076-0.1.0.dist-info/top_level.txt'
adding 'greetlab_25020007076-0.1.0.dist-info/RECORD'
removing build/bdist.macosx-26.0-arm64/wheel
Successfully built greetlab_25020007076-0.1.0.tar.gz and greetlab_25020007076-0.1.0-py3-none-any.whl
(course_env) liuchenxi@liuchenxideMacBook-Air q09 % ls dist
greetlab_25020007076-0.1.0-py3-none-any.whl    greetlab_25020007076-0.1.0.tar.gz
(course_env) liuchenxi@liuchenxideMacBook-Air q09 % python -m venv venv
(course_env) liuchenxi@liuchenxideMacBook-Air q09 % venv/bin/pip install dist/greetlab_25020007076-0.1.0-py3-none-any.whl
1
Looking in indexes: https://mirrors.ustc.edu.cn/pypi/simple/
Processing ./dist/greetlab_25020007076-0.1.0-py3-none-any.whl
Installing collected packages: greetlab-25020007076
Successfully installed greetlab-25020007076-0.1.0
(course_env) liuchenxi@liuchenxideMacBook-Air q09 % cd ..
(course_env) liuchenxi@liuchenxideMacBook-Air w3 % q09/venv/bin/sdt-greet --name 25020007076
Hello, 25020007076!
(course_env) liuchenxi@liuchenxideMacBook-Air w3 %

```

图 1.7: 在 q09 之外运行 sdt-greet

### 1.1.2 实验结果

第九题完成了从源码构建 wheel、在干净虚拟环境中仅从 wheel 安装、并在 q09 之外运行命令行工具验证输出的完整流程, 验证了 Python 打包 (Packaging) 与安装 (Shipping) 的整条链路。

## 1.2 第 10 题让编程智能体进入可验证的修复循环

### 1.2.1 实验内容

如图 1.8 所示, 使用 rsync 将 q09 复制为 q10 (用 `--exclude` 排除 venv、build 等构建产物), 并在 q10 中创建 `test_cli.py`: 当 name 只含空白字符时, main 应以 `SystemExit(2)`

结束。

```
w3 -- zsh -- 120x30
adding 'greetlab_25020007076-0.1.0.dist-info/RECORD'
removing build/bdist.macosx-26.0-arm64/wheel
Successfully built greetlab_25020007076-0.1.0.tar.gz and greetlab_25020007076-0.1.0-py3-none-any.whl
(course_env) liuchenxi@liuchenxiMacBook-Air q09 % ls dist
greetlab_25020007076-0.1.0-py3-none-any.whl  greetlab_25020007076-0.1.0.tar.gz
(course_env) liuchenxi@liuchenxiMacBook-Air q09 % python -m venv venv
(course_env) liuchenxi@liuchenxiMacBook-Air q09 % venv/bin/pip install dist/greetlab_25020007076-0.1.0-py3-none-any.wh
1
Looking in indexes: https://mirrors.ustc.edu.cn/pypi/simple/
Processing ./dist/greetlab_25020007076-0.1.0-py3-none-any.whl
Installing collected packages: greetlab-25020007076
Successfully installed greetlab-25020007076-0.1.0
(course_env) liuchenxi@liuchenxiMacBook-Air q09 % cd ..
(course_env) liuchenxi@liuchenxiMacBook-Air w3 % q09/venv/bin/sdt-greet --name 25020007076
Hello, 25020007076!
(course_env) liuchenxi@liuchenxiMacBook-Air w3 % rsync -a --exclude venv --exclude build --exclude "src/*.egg-info" q0
9/ q10/
(course_env) liuchenxi@liuchenxiMacBook-Air w3 % ls q10
dist  pyproject.toml  src
(course_env) liuchenxi@liuchenxiMacBook-Air w3 % cat > q10/test_cli.py <<'EOF'import pytest
from greetlab.cli import main

def test_blank_name_exits_with_code_2():
    with pytest.raises(SystemExit) as excinfo:
        main(["--name", ""])
    assert excinfo.value.code == 2
EOF
cat q10/test_cli.py
```

图 1.8: 复制 q09 为 q10 并创建 test\_cli.py

如图 1.9 所示，先运行 `PYTHONPATH=src python -m pytest -v` 确认测试失败：报错 `TypeError: main() takes 0 positional arguments but 1 was given`。原因是当前 `main` 函数既不接收参数、也不会校验空白姓名，与测试期望不符，符合“先运行测试确认失败”的要求。

```
q10 -- zsh -- 120x30
(course_env) liuchenxi@liuchenxiMacBook-Air w3 % cd q10
(course_env) liuchenxi@liuchenxiMacBook-Air q10 % PYTHONPATH=src python -m pytest -v
===== test session starts =====
platform darwin -- Python 3.14.7, pytest-9.1.1, pluggy-1.6.0 -- /Users/liuchenxi/Documents/系统开发工具/course_env/bin/p
ython
cachedir: .pytest_cache
rootdir: /Users/liuchenxi/Documents/GitHub/lcx_course_work/w3/q10
configfile: pyproject.toml
collected 1 item

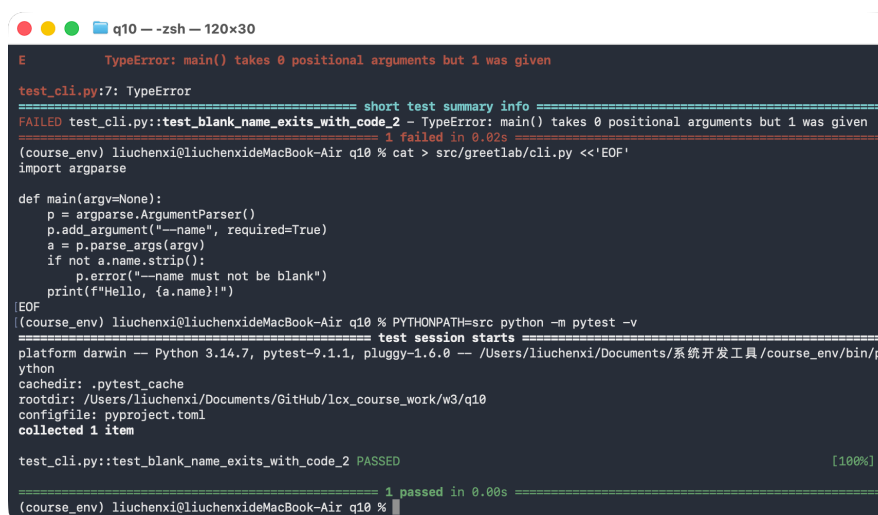
test_cli.py::test_blank_name_exits_with_code_2 FAILED [100%]

===== FAILURES =====
test_cli.py::test_blank_name_exits_with_code_2
def test_blank_name_exits_with_code_2():
    with pytest.raises(SystemExit) as excinfo:
        main(["--name", ""])
>
E       TypeError: main() takes 0 positional arguments but 1 was given

test_cli.py:7: TypeError
===== short test summary info =====
FAILED test_cli.py::test_blank_name_exits_with_code_2 - TypeError: main() takes 0 positional arguments but 1 was given
1 failed in 0.02s
(course_env) liuchenxi@liuchenxiMacBook-Air q10 %
```

图 1.9: 先运行测试确认失败

接下来向编程智能体说明目标（`name` 为纯空白时 `main` 应以 `SystemExit(2)` 结束）、约束（只修改实现、不修改测试）和测试命令，要求它修改实现并运行测试。如图 1.10 所示，智能体把 `main` 改为接收可选 `argv` 参数，并在 `parse` 后校验 `a.name.strip()`，空白时调用 `p.error` 使程序以退出码 2 终止；修改后再次运行测试，输出 1 passed。



```

q10 --zsh -- 120x30
E      TypeError: main() takes 0 positional arguments but 1 was given
test_cli.py:7: TypeError
===== short test summary info =====
FAILED test_cli.py::test_blank_name_exits_with_code_2 - TypeError: main() takes 0 positional arguments but 1 was given
===== 1 failed in 0.02s =====
(course_env) liuchenxi@liuchenxideMacBook-Air q10 % cat > src/greetlab/cli.py <<'EOF'
import argparse

def main(argv=None):
    p = argparse.ArgumentParser()
    p.add_argument("--name", required=True)
    a = p.parse_args(argv)
    if not a.name.strip():
        p.error("--name must not be blank")
    print(f"Hello, {a.name}!")
EOF
(course_env) liuchenxi@liuchenxideMacBook-Air q10 % PYTHONPATH=src python -m pytest -v
===== test session starts =====
platform darwin -- Python 3.14.7, pytest-9.1.1, pluggy-1.6.0 -- /Users/liuchenxi/Documents/系统开发工具/course_env/bin/p
ython
cachedir: .pytest_cache
rootdir: /Users/liuchenxi/Documents/GitHub/lcx_course_work/w3/q10
configfile: pyproject.toml
collected 1 item

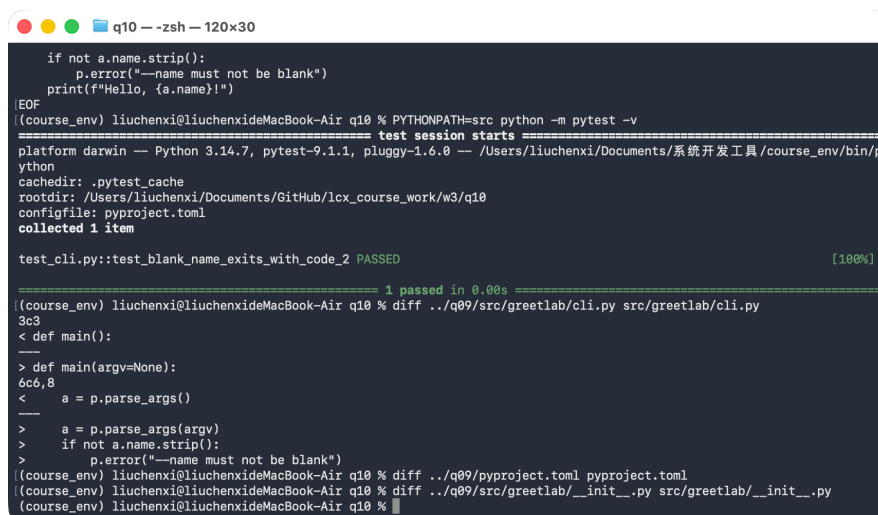
test_cli.py::test_blank_name_exits_with_code_2 PASSED [100%]

===== 1 passed in 0.00s =====
(course_env) liuchenxi@liuchenxideMacBook-Air q10 %

```

图 1.10: 智能体修改 cli.py 后测试通过

如图 1.11 所示, 人工检查 diff: `diff ../q09/src/greetlab/cli.py src/greetlab/cli.py` 显示只有 main 函数签名与空白校验两处预期改动; `pyproject.toml` 和 `__init__.py` 与 q09 完全一致, 说明智能体没有引入无关修改。



```

q10 --zsh -- 120x30
if not a.name.strip():
    p.error("--name must not be blank")
print(f"Hello, {a.name}!")
EOF
(course_env) liuchenxi@liuchenxideMacBook-Air q10 % PYTHONPATH=src python -m pytest -v
===== test session starts =====
platform darwin -- Python 3.14.7, pytest-9.1.1, pluggy-1.6.0 -- /Users/liuchenxi/Documents/系统开发工具/course_env/bin/p
ython
cachedir: .pytest_cache
rootdir: /Users/liuchenxi/Documents/GitHub/lcx_course_work/w3/q10
configfile: pyproject.toml
collected 1 item

test_cli.py::test_blank_name_exits_with_code_2 PASSED [100%]

===== 1 passed in 0.00s =====
(course_env) liuchenxi@liuchenxideMacBook-Air q10 % diff ../q09/src/greetlab/cli.py src/greetlab/cli.py
3c3
< def main():
--
> def main(argv=None):
6c6,8
<     a = p.parse_args()
--
>     a = p.parse_args(argv)
>     if not a.name.strip():
>         p.error("--name must not be blank")
(course_env) liuchenxi@liuchenxideMacBook-Air q10 % diff ../q09/pyproject.toml pyproject.toml
(course_env) liuchenxi@liuchenxideMacBook-Air q10 % diff ../q09/src/greetlab/__init__.py src/greetlab/__init__.py
(course_env) liuchenxi@liuchenxideMacBook-Air q10 %

```

图 1.11: 人工检查 diff 确认无无关修改

如图 1.12 所示, 再次运行测试全部通过; 并用不超过 5 行的 `ai_log.md` 记录核心提示、智能体改动和人工验证三部分内容。

```

q10 --zsh -- 120x30
> if not a.name.strip():
>     p.error("--name must not be blank")
(course_env) liuchenxi@liuchenxideMacBook-Air q10 % diff ../q09/pyproject.toml pyproject.toml
(course_env) liuchenxi@liuchenxideMacBook-Air q10 % diff ../q09/src/greetlab/__init__.py src/greetlab/__init__.py
(course_env) liuchenxi@liuchenxideMacBook-Air q10 % PYTHONPATH=src python -m pytest -v
===== test session starts =====
platform darwin -- Python 3.14.7, pytest-9.1.1, pluggy-1.6.0 -- /Users/liuchenxi/Documents/系统开发工具/course_env/bin/python
cachedir: .pytest_cache
rootdir: /Users/liuchenxi/Documents/GitHub/lcx_course_work/w3/q10
configfile: pyproject.toml
collected 1 item

test_cli.py::test_blank_name_exits_with_code_2 PASSED [100%]

===== 1 passed in 0.00s =====
(course_env) liuchenxi@liuchenxideMacBook-Air q10 % cat > ai_log.md <<'EOF'
# ai_log
1. 核心提示：修复纯空白 --name 仍输出问候并以 0 退出的缺陷；目标 main 对空白姓名以 SystemExit(2) 结束；约束只改变实现不改
测试；测试命令：cd q10 后运行 PYTHONPATH=src python -m pytest -v。
2. 智能体改动：cli.py 的 main 增加 argv 参数，parse 后校验 a.name.strip()，空白时调用 p.error 使退出码为 2。
3. 人工验证：diff 确认仅 cli.py 有预期改动且无无关修改；再次运行 pytest 全部通过。
EOF
(course_env) liuchenxi@liuchenxideMacBook-Air q10 % cat ai_log.md
# ai_log
1. 核心提示：修复纯空白 --name 仍输出问候并以 0 退出的缺陷；目标 main 对空白姓名以 SystemExit(2) 结束；约束只改变实现不改
测试；测试命令：cd q10 后运行 PYTHONPATH=src python -m pytest -v。
2. 智能体改动：cli.py 的 main 增加 argv 参数，parse 后校验 a.name.strip()，空白时调用 p.error 使退出码为 2。
3. 人工验证：diff 确认仅 cli.py 有预期改动且无无关修改；再次运行 pytest 全部通过。
(course_env) liuchenxi@liuchenxideMacBook-Air q10 %

```

图 1.12: 再次运行测试并记录 ai\_log.md

## 1.2.2 实验结果

第十题完成了“测试先行失败、智能体修复、人工检查 diff、再次验证”的可验证修复循环：测试从 1 failed 变为 1 passed，diff 仅含预期改动，ai\_log.md 用 4 行记录完整闭环。

## 1.3 第 11 题把“无法处理”的协作材料改成可执行信息

### 1.3.1 实验内容

围绕“空姓名仍输出问候语”这一缺陷，改写三段质量较差的协作材料。如图 1.13 所示，在 q11 的 communication.md 中重写三部分内容：Issue 包含环境、复现命令、期望结果和实际结果，未知信息写“待确认”；提交信息标题使用祈使语气，正文说明问题和解决方案；评审意见指出具体行为、风险和建议动作，并标注 Blocking、Suggestion 或 Nit。

```

w3 --zsh -- 120x30
## 提交信息
fix: 空白姓名改为报错退出
sdt-greet 对纯空白 --name 仍输出问候并以 0 退出，会被脚本误判为成功。在 main 中校验 name.strip()，空白时调用 p.error 以退出码 2 终止。

## 评审意见
- [Blocking] 空白姓名未校验，输出 "Hello, !" 且退出码 0，调用方无法区分成功与参数错误；建议校验后以退出码 2 拒绝。
- [Suggestion] 错误提示补充期望的合法输入；[Nit] 校验逻辑可抽成独立函数便于单测。
EOF
(course_env) liuchenxi@liuchenxideMacBook-Air w3 % cat q11/communication.md
# communication.md

## Issue
- 环境: macOS, greetlab 0.1.0 (是否仅 Windows 复现: 待确认)
- 复现命令: sdt-greet --name " "
- 期望结果: 拒绝空白姓名，退出码 2
- 实际结果: 输出 "Hello, !"，退出码 0

## 提交信息
fix: 空白姓名改为报错退出
sdt-greet 对纯空白 --name 仍输出问候并以 0 退出，会被脚本误判为成功。在 main 中校验 name.strip()，空白时调用 p.error 以退出码 2 终止。

## 评审意见
- [Blocking] 空白姓名未校验，输出 "Hello, !" 且退出码 0，调用方无法区分成功与参数错误；建议校验后以退出码 2 拒绝。
- [Suggestion] 错误提示补充期望的合法输入；[Nit] 校验逻辑可抽成独立函数便于单测。
(course_env) liuchenxi@liuchenxideMacBook-Air w3 % wc -m q11/communication.md
403 q11/communication.md
(course_env) liuchenxi@liuchenxideMacBook-Air w3 %

```

图 1.13: 重写 communication.md 并核验字数

经 wc -m 核验，初版全文为 403 字，超出 400 字限制。如图 1.14 所示，精简冗余表

述后全文为 384 字，满足“全文控制在 400 字以内”的要求，且保持专业、简洁。



```
w3 --zsh -- 120x30

fix: 空白姓名改为报错退出
sdt-greet 对纯空白 --name 仍输出问候并以 0 退出，会被脚本误判为成功。在 main 中校验 name.strip()，空白时调用 p.error 以退出码 2 终止。

## 评审意见
- [Blocking] 空白姓名未校验，输出 "Hello, !" 且退出码 0，调用方无法区分成功与参数错误；建议校验后以退出码 2 拒绝。
- [Suggestion] 错误提示补充期望的合法输入；[Nit] 校验逻辑可抽成独立函数便于单测。
(course_env) liuchenxi@liuchenxiMacBook-Air w3 % wc -m q11/communication.md
403 q11/communication.md
(course_env) liuchenxi@liuchenxiMacBook-Air w3 %
(course_env) liuchenxi@liuchenxiMacBook-Air w3 % cat > q11/communication.md <<'EOF'
# communication.md

## Issue
- 环境: macOS, greetlab 0.1.0 (是否仅 Windows 复现: 待确认)
- 复现命令: sdt-greet --name ""
- 期望结果: 拒绝空白姓名，退出码 2
- 实际结果: 输出 "Hello, !"，退出码 0

## 提交信息
fix: 空白姓名改为报错退出
sdt-greet 对纯空白 --name 仍输出问候并以 0 退出，易被误判为成功。在 main 校验 name.strip()，空白时以退出码 2 终止。

## 评审意见
- [Blocking] 空白姓名未校验，输出 "Hello, !" 且退出码 0，调用方无法区分成功与参数错误；建议校验后以退出码 2 拒绝。
- [Suggestion] 错误提示补充期望的合法输入；[Nit] 校验逻辑可抽成独立函数便于单测。
EOF
(course_env) liuchenxi@liuchenxiMacBook-Air w3 % wc -m q11/communication.md
384 q11/communication.md
(course_env) liuchenxi@liuchenxiMacBook-Air w3 %
```

图 1.14: 精简后全文 384 字

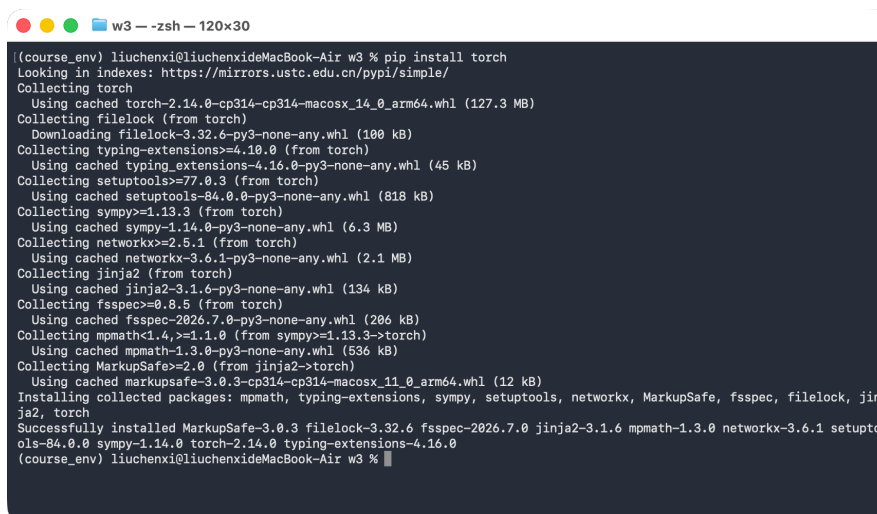
### 1.3.2 实验结果

第十一题最终 communication.md 全文 384 字 ( $\leq 400$ )：Issue 部分补全了环境、复现命令、期望结果和实际结果，并将“是否仅 Windows 复现”标注为待确认；提交信息以 fix：空白姓名改为报错退出为祈使式标题，正文说明了问题与解决方案；评审意见包含 Blocking（未校验导致误判成功）、Suggestion（补充合法输入提示）和 Nit（校验逻辑可独立成函数便于单测）。

## 1.4 第 12 题修复一个可复现的线性回归训练循环

### 1.4.1 实验内容

如图 1.15 所示，在 course\_env 中执行 `pip install torch` 安装 PyTorch(torch 2.14.0)。

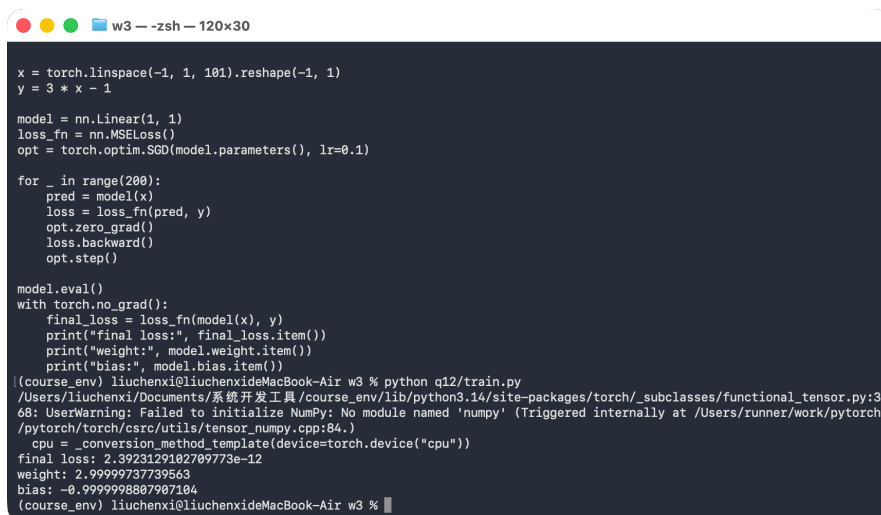


```
w3 --zsh -- 120x30

(course_env) liuchenxi@liuchenxiMacBook-Air w3 % pip install torch
Looking in indexes: https://mirrors.ustc.edu.cn/pypi/simple/
Collecting torch
  Using cached torch-2.14.0-cp314-cp314-macosx_14_0_arm64.whl (127.3 MB)
Collecting filelock (from torch)
  Downloading filelock-3.32.6-py3-none-any.whl (100 kB)
Collecting typing-extensions>=4.10.0 (from torch)
  Using cached typing_extensions-4.16.0-py3-none-any.whl (45 kB)
Collecting setuptools>=77.0.3 (from torch)
  Using cached setuptools-84.0.0-py3-none-any.whl (818 kB)
Collecting sympy>=1.13.3 (from torch)
  Using cached sympy-1.14.0-py3-none-any.whl (6.3 MB)
Collecting networkx>=2.5.1 (from torch)
  Using cached networkx-3.6.1-py3-none-any.whl (2.1 MB)
Collecting jinja2 (from torch)
  Using cached jinja2-3.1.6-py3-none-any.whl (134 kB)
Collecting fsspec>=0.8.5 (from torch)
  Using cached fsspec-2026.7.0-py3-none-any.whl (206 kB)
Collecting mpmath<1.4, >=1.1.0 (from sympy>=1.13.3->torch)
  Using cached mpmath-1.3.0-py3-none-any.whl (536 kB)
Collecting MarkupSafe>=2.0 (from jinja2->torch)
  Using cached MarkupSafe-3.0.3-cp314-cp314-macosx_11_0_arm64.whl (12 kB)
Installing collected packages: mpmath, typing-extensions, sympy, setuptools, networkx, MarkupSafe, fsspec, filelock, jin
ja2, torch
Successfully installed MarkupSafe-3.0.3 filelock-3.32.6 fsspec-2026.7.0 jinja2-3.1.6 mpmath-1.3.0 networkx-3.6.1 setupto
ols-84.0.0 sympy-1.14.0 torch-2.14.0 typing-extensions-4.16.0
(course_env) liuchenxi@liuchenxiMacBook-Air w3 %
```

图 1.15: 安装 PyTorch

如图 1.16 所示, 创建 `q12/train.py`, 在训练循环中补全 `TODO`: 依次调用 `opt.zero_grad()` 清空梯度、`loss.backward()` 反向传播、`opt.step()` 更新参数; 训练结束后调用 `model.eval()` 进入评估模式, 并在 `torch.no_grad()` 中计算并打印最终损失、`weight` 和 `bias`。



```
w3 -- zsh -- 120x30

x = torch.linspace(-1, 1, 101).reshape(-1, 1)
y = 3 * x - 1

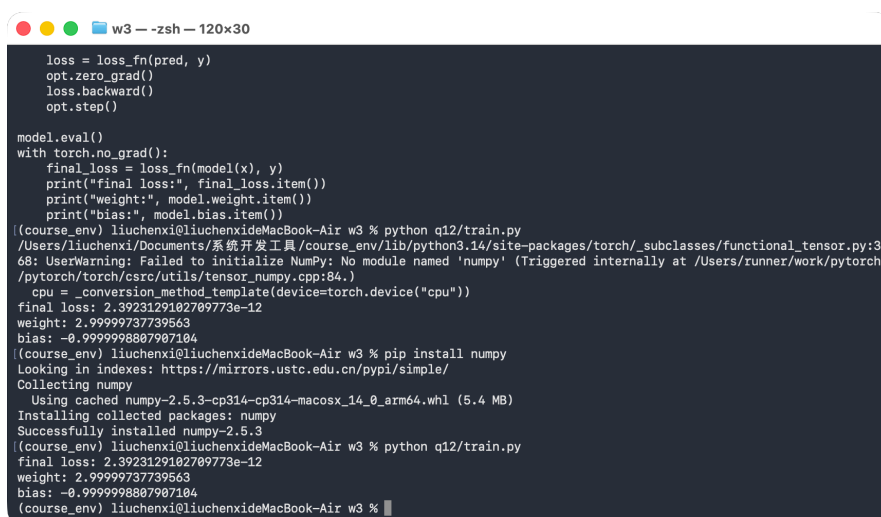
model = nn.Linear(1, 1)
loss_fn = nn.MSELoss()
opt = torch.optim.SGD(model.parameters(), lr=0.1)

for _ in range(200):
    pred = model(x)
    loss = loss_fn(pred, y)
    opt.zero_grad()
    loss.backward()
    opt.step()

model.eval()
with torch.no_grad():
    final_loss = loss_fn(model(x), y)
    print("final loss:", final_loss.item())
    print("weight:", model.weight.item())
    print("bias:", model.bias.item())
(course_env) liuchenxi@liuchenxideMacBook-Air w3 % python q12/train.py
/Users/liuchenxi/Documents/系统开发工具/course_env/lib/python3.14/site-packages/torch/_subclasses/functional_tensor.py:3
68: UserWarning: Failed to initialize NumPy: No module named 'numpy' (Triggered internally at /Users/runner/work/pytorch
/pytorch/torch/csrc/utils/tensor_numpy.cpp:84.)
  cpu = _conversion_method_template(device=torch.device("cpu"))
final loss: 2.3923129102709773e-12
weight: 2.99999737739563
bias: -0.999998807907104
(course_env) liuchenxi@liuchenxideMacBook-Air w3 %
```

图 1.16: 创建 `train.py` 并补全 `TODO` 后运行

首次运行时 `torch` 提示缺少 `numpy`; 如图 1.17 所示, 安装 `numpy` 后重新运行, 输出 `final loss: 2.3923129102709773e-12`、`weight: 2.99999737739563`、`bias: -0.999998807907104`。最终损失远小于题目要求的 0.001, `weight` 收敛到约 3、`bias` 收敛到约 -1, 且全程没有直接把 `weight` 和 `bias` 赋值为 3 和 -1。



```
w3 -- zsh -- 120x30

    loss = loss_fn(pred, y)
    opt.zero_grad()
    loss.backward()
    opt.step()

model.eval()
with torch.no_grad():
    final_loss = loss_fn(model(x), y)
    print("final loss:", final_loss.item())
    print("weight:", model.weight.item())
    print("bias:", model.bias.item())
(course_env) liuchenxi@liuchenxideMacBook-Air w3 % python q12/train.py
/Users/liuchenxi/Documents/系统开发工具/course_env/lib/python3.14/site-packages/torch/_subclasses/functional_tensor.py:3
68: UserWarning: Failed to initialize NumPy: No module named 'numpy' (Triggered internally at /Users/runner/work/pytorch
/pytorch/torch/csrc/utils/tensor_numpy.cpp:84.)
  cpu = _conversion_method_template(device=torch.device("cpu"))
final loss: 2.3923129102709773e-12
weight: 2.99999737739563
bias: -0.999998807907104
(course_env) liuchenxi@liuchenxideMacBook-Air w3 % pip install numpy
Looking in indexes: https://mirrors.ustc.edu.cn/pypi/simple/
Collecting numpy
  Using cached numpy-2.5.3-cp314-cp314-macosx_14_0_arm64.whl (5.4 MB)
Installing collected packages: numpy
Successfully installed numpy-2.5.3
(course_env) liuchenxi@liuchenxideMacBook-Air w3 % python q12/train.py
final loss: 2.3923129102709773e-12
weight: 2.99999737739563
bias: -0.999998807907104
(course_env) liuchenxi@liuchenxideMacBook-Air w3 %
```

图 1.17: 安装 `numpy` 后重新运行 `train.py`

如图 1.18 所示, 最后依次查看各实验目录的文件清单, 确认 `q09-q12` 的实验产物完整。

```
w3 -- -zsh -- 120x30
weight: 2.99999737739563
bias: -0.9999998807907104
(course_env) liuchenxi@liuchenxiMacBook-Air w3 % pip install numpy
Looking in indexes: https://mirrors.ustc.edu.cn/pypi/simple/
Collecting numpy
  Using cached numpy-2.5.3-cp314-cp314-macosx_14_0_arm64.whl (5.4 MB)
Installing collected packages: numpy
Successfully installed numpy-2.5.3
(course_env) liuchenxi@liuchenxiMacBook-Air w3 % python q12/train.py
final loss: 2.3923129102709773e-12
weight: 2.99999737739563
bias: -0.9999998807907104
(course_env) liuchenxi@liuchenxiMacBook-Air w3 % cat q10/test_cli.py
import pytest
from greetlab.cli import main

def test_blank_name_exits_with_code_2():
    with pytest.raises(SystemExit) as excinfo:
        main(["--name", ""])
    assert excinfo.value.code == 2
(course_env) liuchenxi@liuchenxiMacBook-Air w3 % ls q10
ai_log.md      dist          pyproject.toml src          test_cli.py
(course_env) liuchenxi@liuchenxiMacBook-Air w3 % ls q11 q12
q11:
communication.md
q12:
train.py
(course_env) liuchenxi@liuchenxiMacBook-Air w3 %
```

图 1.18: 核实验目录文件清单

## 1.4.2 实验结果

第十二题通过补全训练循环（zero\_grad、backward、step）和评估阶段的 no\_grad 计算，完成了 PyTorch CPU 线性回归训练：最终损失  $2.39 \times 10^{-12} < 0.001$ ，weight 收敛至  $2.999997 \approx 3$ ，bias 收敛至  $-0.9999999 \approx -1$ ，训练成功。

## 1.5 Github 仓库地址

本次课程的报告、练习、代码等，均已上传至个人 github 仓库。  
Github 仓库地址：[https://github.com/lcx821/lcx\\_course\\_work](https://github.com/lcx821/lcx_course_work)

## 课后练习内容和结果

### 2.1 基础命令与重定向

#### 题 1：确认 Shell 环境

**要求：**本课程要求使用类 Unix Shell (Bash 或 ZSH)。运行 `echo $SHELL` 确认。

**解答：**

```
echo $SHELL
```

**实际输出：**

```
/bin/zsh
```

**说明：**当前默认 Shell 是 zsh，属于类 Unix Shell，符合课程要求。bash 也可用（版本 3.2.57）。Windows 用户需使用 WSL 或 Linux 虚拟机，不能用 `cmd.exe` 或 PowerShell。

#### 题 2：ls -l 的输出格式

**要求：**ls -l 的 -l 选项作用是什么？运行 `ls -l /`，每一行最前面的 10 个字符分别代表什么？

**解答：**

```
ls -l /
```

**实际输出（节选）：**

```
total 10
drwxrwxr-x 42 root  admin 1344 Aug 30 13:55 Applications
drwxr-xr-x@ 39 root  wheel 1248 Apr  6 16:10 bin
lrwxr-xr-x@  1 root  wheel  11 Apr  6 16:10 etc -> private/etc
```

**-l 选项作用：**以长格式 (long format) 列出文件，显示文件类型、权限、硬链接数、所有者、所属组、大小、修改时间和文件名。

**前 10 个字符的含义：**

例如 `drwxr-xr-x` 表示：这是一个目录，所有者可读写执行，组用户可读执行，其他用户可读执行。

| 位置       | 含义               | 示例                         |
|----------|------------------|----------------------------|
| 第 1 位    | 文件类型             | d = 目录, l = 符号链接, - = 普通文件 |
| 第 2-4 位  | 所有者 (user) 权限    | rwX = 读、写、执行               |
| 第 5-7 位  | 所属组 (group) 权限   | r-X = 读、执行 (无写权限)          |
| 第 8-10 位 | 其他用户 (others) 权限 | r-X = 读、执行                 |

注：第 11 位可能是 @ (macOS 扩展属性) 或 + (ACL)，不属于权限位。

题 3: glob 模式匹配

要求：什么是 glob？测试 `ls *.txt`、`ls file?.txt`、`ls \{a,b,c\}.txt` 等模式。  
glob 是什么：glob 是 Shell 提供的文件名通配符模式匹配机制。Shell 在执行命令前，会先将包含通配符的参数展开为匹配的文件名列表，再传给程序。

| 模式        | 含义                   |
|-----------|----------------------|
| *         | 匹配任意长度的任意字符（包括空）     |
| ?         | 匹配恰好一个任意字符           |
| [abc]     | 匹配 a、b、c 中任意一个字符     |
| [a-z]     | 匹配 a 到 z 范围内任意一个字符   |
| \{a,b,c\} | 大括号扩展，生成 a、b、c 多个字符串 |

实际测试：

```
mkdir glob_test && cd glob_test
touch file1.txt file2.txt file3.txt a.txt b.txt c.txt abc.txt ab.txt

ls *.txt           # 匹配所有 .txt 文件
ls file?.txt       # 匹配 file + 任意一个字符 + .txt
ls {a,b,c}.txt     # 大括号扩展为 a.txt b.txt c.txt
ls [ab]*.txt       # 匹配以 a 或 b 开头的 .txt 文件
```

输出：

```
=== ls *.txt ===
a.txt ab.txt abc.txt b.txt c.txt file1.txt file2.txt file3.txt

=== ls file?.txt ===
file1.txt file2.txt file3.txt

=== ls {a,b,c}.txt ===
a.txt b.txt c.txt

=== ls [ab]*.txt ===
a.txt ab.txt abc.txt b.txt
```

题 4：三种引号的区别

要求：' 单引号'、" 双引号" 和 \\$\$'ANSI 引号' 有什么区别？写一条命令，输出同时包含字面量 \、! 和换行符的字符串。

| 引号类型            | 变量展开 | 转义序列   | 历史扩展 | 适用场景       |
|-----------------|------|--------|------|------------|
| '...' 单引号       | 否    | 否      | 否    | 纯字面量字符串    |
| "..." 双引号       | 是    | 否（除少数） | 可能触发 | 需要嵌入变量     |
| \\$'...' ANSI-C | 否    | 是      | 否    | 需要转义序列的字面量 |

实际测试：

```
# 单引号：完全字面量
echo 'It costs $100! Hello\nWorld'
# 输出：It costs $100! Hello\nWorld

# 双引号：展开变量，\n 不解释
price=100
echo "It costs $price! Hello\nWorld"
# 输出：It costs $price! Hello\nWorld

# ANSI-C 引号：解释 \n, $ 是字面量
echo '$It costs $100! Hello\nWorld'
# 输出：
# It costs $100! Hello
# World
```

题目要求的命令（同时包含字面量 \、! 和换行符）：

```
echo '$价格是 $100!\n这是第二行'
```

输出：

```
价格是 $100!
这是第二行
```

也可以用 `printf '价格是 \n这是第二行'`，`printf` 默认就解释转义序列。

题 5：标准流与重定向

要求：Shell 有三条标准流：stdin(0)、stdout(1)、stderr(2)。运行 `ls /nonexistent /tmp`，把 stdout 和 stderr 分别重定向到两个文件。如何把两者都重定向到同一个文件？

三条标准流：

- **stdin (0)**：标准输入，默认来自键盘
- **stdout (1)**：标准输出，默认打印到终端
- **stderr (2)**：标准错误，默认也打印到终端（独立于 stdout）

分别重定向到两个文件：

```
ls /nonexistent /tmp > stdout.txt 2> stderr.txt
```

- `> stdout.txt` 等价于 `1> stdout.txt`，把标准输出写入 `stdout.txt`
- `2> stderr.txt` 把标准错误写入 `stderr.txt`

实际输出：

```
stdout.txt: 包含 /tmp 目录下的文件列表
stderr.txt: 包含 "ls: /nonexistent: No such file or directory"
```

两者重定向到同一个文件（两种方法）：

方法 1（POSIX 标准，推荐）：

```
ls /nonexistent /tmp > both.txt 2>&1
```

- `> both.txt` 先把 `stdout` 指向 `both.txt`
- `2>&1` 把 `stderr` 重定向到当前 `stdout` 指向的地方（即 `both.txt`）
- 顺序很重要：`2>&1` 必须在 `>` 之后

方法 2（bash/zsh 简写）：

```
ls /nonexistent /tmp &> both.txt
```

## 题 6：退出状态与 `\&\& / ||`

要求：`\$?` 保存上一条命令的退出状态（0=成功）。`\&\&` 仅在前一条成功时执行后一条；`||` 仅在前一条失败时执行后一条。写一行命令：仅当 `/tmp/mydir` 不存在时才创建它。

退出状态：每个命令执行后返回一个 0-255 的整数，0 表示成功，非 0 表示失败。可用 `echo \$?` 查看。

逻辑运算符：

- `cmd1 \&\& cmd2`：`cmd1` 成功（退出码 0）才执行 `cmd2`
- `cmd1 || cmd2`：`cmd1` 失败（退出码非 0）才执行 `cmd2`
- 两者可组合：`cmd1 \&\& cmd2 || cmd3`（类似三元运算符）

题目要求的命令：

```
[ -d /tmp/mydir ] || mkdir /tmp/mydir
```

解释：

- `[ -d /tmp/mydir ]` 测试目录是否存在（`test -d` 的简写）
- 如果存在，`[` 命令返回 0（成功），`||` 后面的 `mkdir` 不执行
- 如果不存在，`[` 返回非 0（失败），`||` 触发 `mkdir /tmp/mydir`

实际验证：

```
# 第一次运行：目录不存在，创建成功
[ -d /tmp/mydir ] || mkdir /tmp/mydir
ls -ld /tmp/mydir
# 输出：drwxr-xr-x  2 liuchenxi  wheel  64 Aug 30 16:51 /tmp/mydir

# 第二次运行：目录已存在，不重复创建
[ -d /tmp/mydir ] || mkdir /tmp/mydir
# 无输出，无报错
```

更简洁的等价写法: `mkdir -p /tmp/mydir` (`-p` 表示目录已存在时不报错), 但题目要求用 `\&\&///` 逻辑。

## 2.2 脚本编写与权限

### 题 7: 为什么 `cd` 必须是 Shell 内建命令?

**要求:** 为什么 `cd` 必须是 Shell 内建命令, 而不能是独立程序? (提示: 想想子进程能影响和不能影响父进程的哪些状态。)

**核心原因:** 子进程无法修改父进程的工作目录。

**详细解释:**

1. **进程独立性:** 在 Unix 系统中, 每个进程都有自己独立的「当前工作目录」(`cwd`)。当 Shell 执行一个外部程序时, 会通过 `fork()` 创建子进程, 再用 `exec()` 加载程序。子进程继承父进程的 `cwd`, 但这只是一个副本。
2. **如果 `cd` 是独立程序:**
  - Shell `fork` 出子进程
  - 子进程执行 `cd /some/path`, 修改的是子进程自己的 `cwd`
  - 子进程结束后, Shell (父进程) 的 `cwd` 完全没有改变
  - 结果就是: 你执行了 `cd /some/path`, 但提示符显示的当前目录没变
3. **内建命令的优势:** 内建命令由 Shell 自身执行, 不创建子进程, 可以直接修改 Shell 进程自己的 `cwd`, 从而真正改变后续命令的工作目录。

**验证:**

```
which cd
# 输出: cd: shell built-in command (或 no cd in $PATH)
# 说明 cd 不是独立程序, 而是 Shell 内建命令
```

**同类内建命令:** `cd`、`export`、`alias`、`exit`、`source/.` 等——凡是需要修改 Shell 自身状态的命令, 都必须是内建命令。

### 题 8: 写脚本检查文件是否存在

**要求:** 写一个脚本, 接收文件名参数 (`\$1`), 用 `test -f` 或 `[ -f ... ]` 检查该文件是否存在, 并根据结果输出不同提示。

**脚本 `check.sh`:**

```
#!/bin/bash
# 检查文件是否存在的脚本

if [ -f "$1" ]; then
    echo "文件 '$1' 存在。"
else
    echo "文件 '$1' 不存在。"
fi
```

**关键点：**

- `\$1` 是第一个命令行参数
- `[ -f "$1" ]` 测试 `\$1` 是否为存在的普通文件 (`-f = file`)
- `"$1"` 加双引号防止文件名含空格时出错
- `if ...; then ... else ... fi` 是 `bash` 条件语句结构

**其他常用测试条件：**

| 条件                   | 含义        |
|----------------------|-----------|
| <code>-f file</code> | 存在且是普通文件  |
| <code>-d file</code> | 存在且是目录    |
| <code>-e file</code> | 存在（任何类型）  |
| <code>-r file</code> | 存在且可读     |
| <code>-w file</code> | 存在且可写     |
| <code>-x file</code> | 存在且可执行    |
| <code>-s file</code> | 存在且大小大于 0 |

**题 9：chmod +x 的作用**

**要求：**把上一题的脚本保存为 `check.sh`。先运行 `./check.sh somefile`，会发生什么？然后执行 `chmod +x check.sh` 再试一次。为什么这一步是必须的？

**chmod 前运行：**

```
./check.sh somefile
# 输出: bash: ./check.sh: Permission denied
```

**chmod 前的权限：**

```
-rw----- 1 liuchenxi staff 147 Aug 30 16:52 check.sh
```

`rw-` = 所有者可读、可写，但不可执行

**执行 chmod +x check.sh 后：**

```
-rwx--x--x 1 liuchenxi staff 147 Aug 30 16:52 check.sh
```

`rwx` = 所有者可读、可写、可执行

**chmod 后运行：**

```
./check.sh somefile
# 输出: 文件 'somefile' 不存在。

./check.sh check.sh
# 输出: 文件 'check.sh' 存在。
```

**为什么必须 chmod +x：**

- Unix 系统中，文件能否被直接执行取决于可执行权限位（`x` 位），而不是文件扩展名
- 新建文件默认没有执行权限（安全考虑）
- `chmod +x` 给所有用户添加执行权限，Shell 才能通过 `./check.sh` 直接运行它
- 没有执行权限时，仍可通过解释器显式运行：`bash check.sh somefile`

shebang 行（`#!/bin/bash`）只在文件有执行权限且被直接运行时才生效。

题 10: `set -x` 的作用

要求：在脚本的 `set` 选项里加入 `-x` 会发生什么？写个简单脚本试试。

`set -x` 的作用：开启调试模式（`xtrace`）。Shell 在执行每条命令前，会先把该命令及其展开后的参数打印到 `stderr`（通常以 `+` 开头），方便追踪脚本执行流程。

演示脚本 `debug\_demo.sh`：

```
#!/bin/bash
set -x

echo "开始执行脚本"
name="Shell"
echo "你好, $name"
for i in 1 2 3; do
    echo "第 $i 次循环"
done
echo "脚本结束"
```

运行输出（`+` 开头的行是 `set -x` 打印的追踪信息）：

```
+ echo '开始执行脚本'
开始执行脚本
+ name=Shell
+ echo '你好, Shell'
你好, Shell
+ i=1
+ echo '第 1 次循环'
第 1 次循环
+ i=2
+ echo '第 2 次循环'
第 2 次循环
+ i=3
+ echo '第 3 次循环'
第 3 次循环
+ echo '脚本结束'
脚本结束
```

常用 `set` 选项总结：

| 选项                           | 作用                   |
|------------------------------|----------------------|
| <code>set -e</code>          | 任何命令失败时立即退出脚本        |
| <code>set -u</code>          | 使用未定义变量时报错（而非当作空字符串） |
| <code>set -o pipefail</code> | 管道中任一命令失败，整个管道视为失败   |
| <code>set -x</code>          | 打印每条执行的命令（调试用）       |

推荐组合：`set -euo pipefail`（脚本开头的「严格模式」），调试时再加 `-x`。

## 题 11：带日期的备份文件

**要求：**写一条命令，把文件复制为带当天日期的备份文件名（例如 `notes.txt` → `notes\_2026-01-12.txt`）。提示：`\$(date +%Y-%m-%d)`

**命令：**

```
cp notes.txt "notes_$(date +%Y-%m-%d).txt"
```

**原理：**

- `\$(date +%Y-%m-%d)` 是**命令替换**：执行 `date` 命令，用其输出替换整个 `\$( )`
- `+%Y-%m-%d` 是 `date` 的格式参数：`%Y` = 四位年，`%m` = 两位月，`%d` = 两位日
- 双引号确保日期展开后作为一个整体参数

**实际运行：**

```
echo "这是笔记内容" > notes.txt
cp notes.txt "notes_$(date +%Y-%m-%d).txt"
ls -l notes*.txt
```

**输出：**

```
-rw-r--r--  1 staff  19 Aug 30 16:52 notes.txt
-rw-r--r--  1 staff  19 Aug 30 16:52 notes_2026-08-30.txt
```

**通用公式：**

```
cp "文件名" "文件名_$(date +%Y-%m-%d).扩展名"
```

**其他常用日期格式：**

```
date +%Y%m%d           # 20260830
date +%Y-%m-%d_%H%M%S  # 2026-08-30_165200（带时间）
date -v-1d +%Y-%m-%d   # 昨天的日期（macOS）
```

## 题 12：修改 flaky test 脚本接收命令行参数

**要求：**修改讲义中的「复现偶尔才会失败的测试」脚本，使它能够从命令行参数接收测试命令，而不是写死 `cargo test my\_test`。提示：`\$1` 或 `\$@`

**修改后的脚本 `flaky\_test.sh`：**

```
#!/bin/bash
set -euo pipefail

# 从命令行参数接收测试命令，而不是写死
TEST_CMD="$@"

if [ -z "$TEST_CMD" ]; then
    echo "用法：$0 <测试命令>"
    echo "示例：$0 cargo test my_test"
    exit 1
fi
```

```
# Start CPU stress in background
stress --cpu 8 &
STRESS_PID=$!

# Setup log file
LOGFILE="test_runs_$(date +%s).log"
echo "Logging to $LOGFILE"
echo "运行测试命令: $TEST_CMD"

# Run tests until one fails
RUN=1
while $TEST_CMD > "$LOGFILE" 2>&1; do
    echo "Run $RUN passed"
    ((RUN++))
done

# Cleanup and report
kill $STRESS_PID
echo "Test failed on run $RUN"
echo "Last 20 lines of output:"
tail -n 20 "$LOGFILE"
echo "Full log: $LOGFILE"
```

关键改动:

- 原脚本写死: while cargo test my\\_test > \"\$LOGFILE\" 2>&1; do
- 改为: TEST\\_CMD=\"\$@\" 接收所有命令行参数, 然后 while \"\$TEST\\_CMD\" > ...

特殊参数说明:

| 参数    | 含义                              |
|-------|---------------------------------|
| \\$1  | 第一个参数                           |
| \\$@  | 所有参数, 每个参数独立 (加引号后\"\$@\" 保留空格) |
| \\$*  | 所有参数, 用 IFS 连接成一个字符串            |
| \\$## | 参数个数                            |
| \\$0  | 脚本自身名称                          |
| \\$!  | 最近一个后台进程的 PID                   |

使用方式:

```
./flaky_test.sh cargo test my_test      # 传 cargo test 命令
./flaky_test.sh pytest tests/           # 传 pytest 命令
./flaky_test.sh false                   # 用 false 测试 (第一次就失败)
```

注意: \"\$@\" 比 \"\$1\" 更适合接收完整命令, 因为命令可能包含多个参数 (如 cargo test my\\_test 是三个参数)。

## 2.3 管道、工具与进阶

### 题 13: 找出 home 目录最常见的 5 种文件扩展名

要求: 使用管道找出 home 目录中最常见的 5 种文件扩展名。提示: 组合 `find`、`grep`/`sed`/`awk`、`sort`、`uniq -c`、`head`

命令:

```
find ~ -type f 2>/dev/null \
| grep -oE '\.[^./]+$' \
| sort \
| uniq -c \
| sort -rn \
| head -n 5
```

逐步拆解:

1. `find ~ -type f`: 递归列出 home 目录下所有普通文件
2. `2>/dev/null`: 把权限错误等 `stderr` 丢弃
3. `grep -oE '\textbackslash .[^{}/.]+\textbackslash $'`: 提取每行末尾的扩展名
  - `-o`: 只输出匹配部分
  - `-E`: 使用扩展正则
  - `\textbackslash .[^{}/.]+\textbackslash $`: 匹配. + 一个或多个非. 非/ 字符 + 行尾
4. `sort`: 排序 (相同扩展名排在一起)
5. `uniq -c`: 去重并统计每行出现次数
6. `sort -rn`: 按数字逆序排序 (出现次数多的在前)
7. `head -n 5`: 取前 5 个

实际输出:

```
11425 .py
9438 .js
6911 .pyc
5855 .md
3948 .json
```

等价的 `awk` 版本:

```
find ~ -type f 2>/dev/null | awk -F. '{print "}.${NF}' | sort | uniq -c | sort -rn |
head -5
```

### 题 14: `xargs` 结合 `find` 统计 `.sh` 文件行数

要求: `xargs` 把 `stdin` 的每一行转换为命令参数。结合 `find` 和 `xargs` (不要用 `find -exec`)，找出目录中所有 `.sh` 文件，并用 `wc -l` 统计每个文件行数。加分项: 正确处理文件名中的空格。提示: `-print0` 和 `-0`

基础版本:

```
find . -name "*.sh" | xargs wc -l
```

**问题：**如果文件名包含空格（如 `my script.sh`），`xargs` 会把它拆成两个参数 `my` 和 `script.sh`，导致错误。

**正确处理空格的版本（加分项）：**

```
find . -name "*.sh" -print0 | xargs -0 wc -l
```

**原理：**

- `find -print0`：用 **null 字符（\0）** 分隔文件名，而不是换行符
- `xargs -0`：以 **null 字符** 作为分隔符读取输入
- 因为文件名中不可能包含 **null 字符**，所以这种方式 **100% 安全** 处理任何文件名（含空格、换行、特殊字符）

**实际输出：**

```
1 ./dir with spaces/my script.sh
34 ./flaky_test.sh
8 ./check.sh
10 ./debug_demo.sh
53 total
```

**xargs 常用参数：**

| 参数    | 含义                                           |
|-------|----------------------------------------------|
| -0    | 以 null 字符分隔输入（配合 <code>find -print0</code> ） |
| -n N  | 每次传递 N 个参数给命令                                |
| -I {} | 用 {} 作为参数占位符                                 |
| -P N  | 并行运行 N 个进程                                   |

## 题 15: curl 获取课程网站 HTML，统计列出了多少讲

**要求：**使用 `curl` 获取课程网站的 HTML，通过 `grep` 统计列出了多少讲。提示：找出每讲课程名称在 HTML 中的共性；用 `curl -s` 关闭进度输出。

**命令：**

```
curl -s https://missing-semester-cn.github.io/2026/ \
| grep -oE 'href="/2026/[a-z-]+/' \
| sort -u \
| wc -l
```

**逐步拆解：**

1. `curl -s URL`：静默模式获取网页 HTML（不显示进度条）
2. `grep -oE 'href="/2026/[a-z-]+/'`：提取所有课程链接
  - 课程链接的共性：`href="/2026/课程名/`
  - 课程名由小写字母和连字符组成
3. `sort -u`：排序并去重（`-u = unique`）
4. `wc -l`：统计行数，即课程讲数

实际输出：

```
9
```

列出各讲名称：

```
curl -s https://missing-semester-cn.github.io/2026/ \
| grep -oE 'href="/2026/[a-z-]+/"' \
| sort -u \
| sed 's|href="/2026/||;s|/||'
```

输出：

```
agentic-coding
beyond-code
code-quality
command-line-environment
course-shell
debugging-profiling
development-environment
shipping-code
version-control
```

共 9 讲，与课程介绍中「九场 1 小时的讲座」一致。

## 题 16: jq 处理 JSON

要求：用 curl 获取示例数据，再用 jq 提取 version 大于 6 的人员姓名。提示：先 jq . 看结构；再试 jq '.[0] | select(...)' | .name'

获取数据并查看结构：

```
curl -s https://microsoftedge.github.io/Demos/json-dummy-data/64KB.json > data.json
jq '.[0]' data.json
```

JSON 结构：

```
{
  "name": "Adeel Solangi",
  "language": "Sindhi",
  "id": "V590F92YF627HFY0",
  "bio": "Donec lobortis eleifend...",
  "version": 6.1
}
```

顶层是一个数组，每个元素包含 name、language、id、bio、version 字段。

提取 version > 6 的人员姓名：

```
jq '.[0] | select(.version > 6) | .name' data.json
```

jq 语法拆解：

- .[]：遍历数组中的每个元素
- select(.version > 6)：筛选 version 字段大于 6 的元素
- .name：提取 name 字段

统计数量：

```
jq '[.[] | select(.version > 6)] | length' data.json
# 输出：89
```

部分输出：

```
"Adeel Solangi"
"Aamir Solangi"
"Adil Eli"
"George Mifsud"
"John Falzon"
...
```

jq 常用操作速查：

| 表达式          | 含义             |
|--------------|----------------|
| .            | 整个输入           |
| .field       | 取字段            |
| .[]          | 遍历数组           |
| select(cond) | 筛选满足条件的元素      |
| length       | 数组长度或字符串长度     |
| keys         | 对象的所有键         |
| map(expr)    | 对数组每个元素应用 expr |

题 17：awk 按列过滤并交换列

要求：写一个 awk 命令：只输出第二列大于 100 的行，并交换第一列和第三列。测试数据：printf 'a 50 x\textrightarrow nb 150 y\textrightarrow nc 200 z\textrightarrow n'

命令：

```
printf 'a 50 x\rightarrow nb 150 y\rightarrow nc 200 z\rightarrow n' \
| awk '$2 > 100 { temp=$1; $1=$3; $3=temp; print }'
```

awk 语法拆解：

- \rightarrow > 100：模式（条件），只处理第二列大于 100 的行
- \{ temp=\rightarrow; \rightarrow=\rightarrow; \rightarrow=temp; print \}：动作
  - temp=\rightarrow：把第一列存到临时变量
  - \rightarrow=\rightarrow：第一列替换为第三列的值
  - \rightarrow=temp：第三列替换为原来的第一列
  - print：打印修改后的整行

实际输出：

```
y 150 b
z 200 c
```

验证：

- 原始行 a 50 x：第二列 50，不大于 100，被过滤
- 原始行 b 150 y：第二列 150 > 100，交换第一列 (b) 和第三列 (y) → y 150 b
- 原始行 c 200 z：第二列 200 > 100，交换第一列 (c) 和第三列 (z) → z 200 c

awk 核心概念：

- awk 按行处理，每行自动按空白字符分成列
- \$1, \$2, \$3... 引用各列，\$0 表示整行
- NR = 当前行号，NF = 当前行列数
- FS = 输入分隔符（默认空白），-F, 可设为逗号
- BEGIN {} 在处理前执行，END {} 在处理后执行

题 18：拆解 SSH 日志管道 + 从历史记录找出最常用命令

要求：拆解讲义中的 SSH 日志处理管道：每一步分别做了什么？然后仿照它构建一个管道，从 ~/.bash\_history（或 ~/.zsh\_history）中找出最常使用的 Shell 命令。

第一部分：拆解 SSH 日志管道

原始管道：

```
ssh myserver 'journalctl -u sshd -b-1 | grep "Disconnected from" \
| sed -E 's/.*Disconnected from .* user (.*?) [^ ]+ port.*/\1/' \
| sort | uniq -c \
| sort -nk1,1 | tail -n10 \
| awk '{print $2}' | paste -sd,
```

逐步拆解：

| 步骤 | 命令                               | 作用                       |
|----|----------------------------------|--------------------------|
| 1  | ssh myserver '...'               | 登录远程服务器，在远程执行引号内的命令      |
| 2  | journalctl -u sshd -b-1          | 查看 sshd 服务在上一次启动时的系统日志   |
| 3  | grep "Disconnected from"         | 筛选包含「断开连接」的日志行           |
| 4  | sed -E 's/...user (.*?) .../\1/' | 用正则提取用户名，(.*?) 捕获，\ 1 输出 |
| 5  | sort                             | 对用户名排序（相同名字排在一起）         |
| 6  | uniq -c                          | 去重并统计每个用户名出现次数           |
| 7  | sort -nk1,1                      | 按第一列（次数）数值排序（升序）         |
| 8  | tail -n10                        | 取最后 10 个，即出现次数最多的 10 个   |
| 9  | awk '{print \$2}'                | 只输出第二列（用户名），去掉次数         |
| 10 | paste -sd,                       | 把多行合并为一行，用逗号分隔           |

最终输出示例：

```
postgres,mysql,oracle,dell,ubuntu,inspur,test,admin,user,root
```

## 第二部分：从历史记录找出最常用命令

命令 (zsh 用户):

```
LC_ALL=C awk '{print $1}' ~/.zsh_history | sort | uniq -c | sort -rn | head -n 10
```

命令 (bash 用户):

```
awk '{print $1}' ~/.bash_history | sort | uniq -c | sort -rn | head -n 10
```

逐步拆解:

1. `awk '\{print \$1\}'`: 提取每行的第一个词 (即命令名, 去掉参数)
2. `sort`: 排序
3. `uniq -c`: 统计每个命令出现次数
4. `sort -rn`: 按次数逆序排序
5. `head -n 10`: 取前 10 个

实际输出 (zsh\_history):

```
123 brew
25 cd
22 git
20 sudo
14 ls
9 echo
8 fc-match
7 which
7 find
6 mkdir
```

说明:

- zsh 的历史文件默认格式可能包含时间戳前缀 ( : 1234567890:0;command ), 如果有, 需要先用 `sed 's/\~{}: [0-9]*:[0-9]*;/'` 去掉前缀
- `LC_ALL=C` 防止遇到非 UTF-8 字节时 `awk/sed` 报错
- 最常用的命令是 `brew` (macOS 包管理器), 其次是 `cd`、`git`、`sudo`、`ls`

## 解题感悟

第三周的实验围绕 Python 打包发布、智能体协作和机器学习训练展开，相比前两周的 Shell 命令和调试，这周的实验明显更接近真实的工程开发流程，也更有挑战。

第 9 题让我第一次完整走了一遍“源码 - 构建 - 安装 - 运行”的打包流程。一开始对 `pyproject.toml` 里的 `[build-system]`、`[project]`、`[project.scripts]` 这些配置完全不熟悉，不知道 `sdt-greet` 这个命令是从哪里来的。后来才明白 `[project.scripts]` 就是把包里的函数注册成命令行入口，构建之后 `pip` 安装，系统就能直接调用。第 9 题要求“只从 `wheel` 安装”，我一开始想在干净环境里直接 `pip install greetlab`，结果当然装不上，后来才理解题目的用意：只有把构建产物本身装进去，才能证明打包是正确的。

第 10 题是让我感触最深的一题。题目要求让编程智能体进入“可验证的修复循环”：先写测试确认失败，再让智能体修复，最后人工检查 `diff`。我一开始是让智能体直接改代码，改完就跑，结果代码虽然能运行，但我自己都不知道改了什么、为什么改。按照题目的流程做下来，先看到 `TypeError` 的失败输出，再看到智能体补上 `argv` 参数和空白校验，最后用 `diff` 确认只有 `cli.py` 有改动，整个过程每一步都有证据，非常踏实。我也第一次意识到，测试不是“多写的作业”，而是让修改可验证的锚点；`ai_log.md` 用几行字把提示、改动和验证记录下来的习惯也值得坚持。

第 11 题字数限制 400 字看起来不难，真正写的时候才发现把问题讲清楚还要控制字数并不容易。我一开始写 `Issue` 的时候把复现步骤写得过于冗长，又漏掉了环境信息，初版写完是 403 字还超了。精简的时候学会了把“环境、复现、期望、实际”这类信息用最少的词说清楚，把未知信息诚实地标注成“待确认”而不是编一个结论。这让我意识到，协作材料最重要的是让接收方“能直接执行”，而不是堆字数。

第 12 题是第一次用 `PyTorch` 写训练循环。题目里把 `zero_grad`、`backward`、`step` 都留成了 `TODO`，补全本身不难，难的是理解为什么要 `zero_grad`：不清空梯度的话，多次 `backward` 会把梯度累加，参数更新就会出错。运行出 `final loss: 2.39e-12` 的时候很有成就感，也体会到机器学习调试“看指标、找原因、改逻辑”的基本思路。

这周的实验比前两周更“工程化”：打包、测试、协作、训练，每一步都有明确的产物和验证标准。虽然过程依然有卡壳的时候（比如环境变量没生效、`numpy` 缺失的警告），但按照“先验证、再修改、最后核对”的流程，问题都能定位到具体环节。希望在后面的课程里能把这种工作方式保持下去。

